CS3807 – Deep Learning Laboratory

Experiment 1: Implementation of a Single Layer Perceptron for Binary Classification

B.Tech Artificial Intelligence & Data Science
Shiv Nadar University Chennai

AY 2026–27

1 Objective

The goal of this experiment is to build a proper understanding of how an artificial neuron works and to implement a Single Layer Perceptron completely from scratch for a binary classification problem. Along the way the idea is to also learn the perceptron learning rule, see why an activation function is needed at all, look at how the model learns over epochs, and finally test the classifier on a real dataset instead of a toy one.

2 Background Theory

An artificial neuron is basically the smallest computational unit of a neural network. It takes in a set of input features, multiplies each of them with a corresponding weight, adds a bias term to the result, and then passes this combined value through an activation function to produce the final output.

2.1 Weighted Sum

$$z = w^T x + b \tag{1}$$

where x is the input vector, w is the weight vector, b is the bias, and z is the resulting linear combination.

2.2 Prediction

$$\hat{y} = f(z) \tag{2}$$

2.3 Weight Update Rule

$$w_{new} = w_{old} + \eta(y - \hat{y})\, x \tag{3}$$

2.4 Bias Update Rule

$$b_{new} = b_{old} + \eta(y - \hat{y}) \tag{4}$$

Here η is the learning rate that controls how big a step is taken during each weight update.

3 Activation Functions

The activation function is what actually decides the output of a neuron once the weighted sum has been computed. It brings in non-linearity into the network, which is what allows deep networks to eventually model complicated, non-linear relationships. In this particular experiment, however, only the Step Activation Function is used, since the classical perceptron was originally designed around it.

3.1 Step Activation Function

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases} \tag{5}$$

Because the Step function only ever outputs a 0 or a 1, it can only be used for problems that are linearly separable, which is exactly the case with the classical perceptron.

Table 1: Common Activation Functions

Activation	**Output Range**	**Typical Application**
Step	$\{0, 1\}$	Classical Perceptron
Sigmoid	$(0, 1)$	Binary Classification Output Layer
Tanh	$(-1, 1)$	Hidden Layers (Traditional Networks)
ReLU	$[0, \infty)$	Hidden Layers in CNNs / MLPs
Leaky ReLU	$(-\infty, \infty)$	Avoiding Dying ReLU
Softmax	$(0, 1)$	Multi-class Classification Output Layer

4 Dataset Description

The dataset used in this experiment is the **Banknote Authentication Dataset** from the UCI Machine Learning Repository (https://archive.ics.uci.edu/dataset/267/banknote+authentication). The features were derived using a Wavelet Transform applied to images of genuine and forged banknotes.

Table 2: Dataset Summary

Instances	1372
Features	4 (Variance, Skewness, Curtosis, Entropy)
Classes	2 (0 – Authentic, 1 – Forged)
Missing Values	None
Task	Binary Classification

5 Experimental Procedure

5.1 Task 1: Dataset Exploration

The dataset was loaded using `pandas`, and the standard exploration steps were carried out: viewing the first five rows, checking the shape of the dataframe, confirming there were no missing values, and looking at the descriptive statistics.

```
import pandas as pd

df = pd.read_csv("data_banknote_authentication.txt", header=None,
```

```
                    names=["Variance","Skewness","Curtosis","Entropy","
                        Class"])

df.head()
df.shape
df.info()
df.describe()
df.isnull().sum()
```

The dataset was confirmed to have 1372 rows and 5 columns (4 features + 1 target), with no missing values in any column. Table 3 shows the descriptive statistics obtained.

Table 3: Descriptive Statistics of the Dataset

	Variance	**Skewness**	**Curtosis**	**Entropy**
mean	0.4337	1.9224	1.3976	-1.1917
std	2.8428	5.8690	4.3100	2.1010
min	-7.0421	-13.7731	-5.2861	-8.5482
25%	-1.7730	-1.7082	-1.5750	-2.4135
50%	0.4962	2.3197	0.6166	-0.5867
75%	2.8215	6.8146	3.1793	0.3948
max	6.8248	12.9516	17.9274	2.4495

5.2 Task 2: Exploratory Data Analysis

The four mandatory plots for EDA (histograms, correlation heatmap, scatter plot and boxplots) were generated using `matplotlib` and `seaborn`, on top of the dataframe loaded in Task 1.

```
import matplotlib.pyplot as plt
import seaborn as sns

# Histograms
fig, axes = plt.subplots(2, 2, figsize=(9, 7))
cols = ["Variance", "Skewness", "Curtosis", "Entropy"]
for ax, c in zip(axes.ravel(), cols):
    ax.hist(df[c], bins=30, color="#4C72B0", edgecolor="black", alpha
        =0.8)
    ax.set_title(c)
plt.tight_layout()
plt.show()

# Correlation heatmap
corr = df.corr()
sns.heatmap(corr, annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Correlation Heatmap")
plt.show()

# Scatter plot (Variance vs Skewness, coloured by class)
colors = df["Class"].map({0: "green", 1: "red"})
plt.scatter(df["Variance"], df["Skewness"], c=colors, alpha=0.6)
plt.xlabel("Variance"); plt.ylabel("Skewness")
plt.title("Scatter Plot: Variance vs Skewness")
plt.show()

# Boxplots
df[cols].plot(kind="box")
```

```
28 plt.title("Boxplots of Features")
29 plt.show()
```

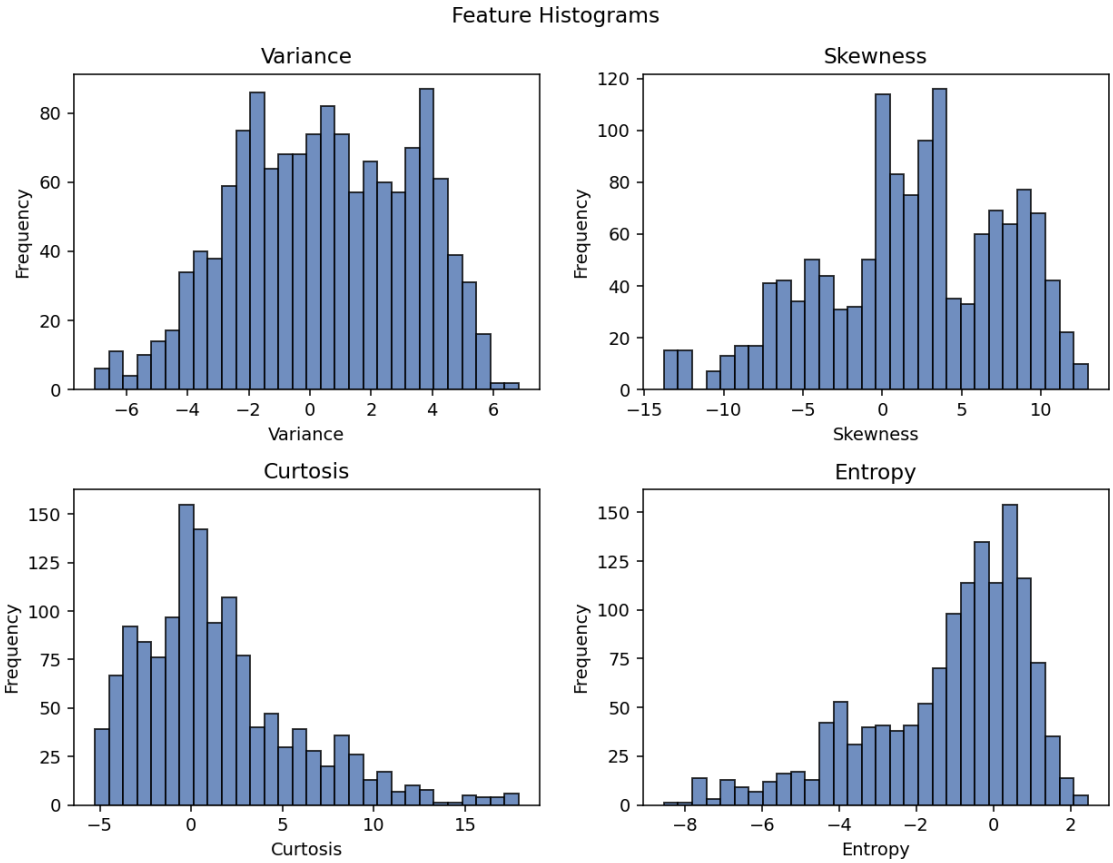

Figure 1: Feature Histograms

Interpretation: Variance and Entropy look reasonably close to a normal-ish spread, whereas Skewness and Curtosis are noticeably more spread out with a longer tail on one side, indicating a few extreme values in the wavelet-transformed features.

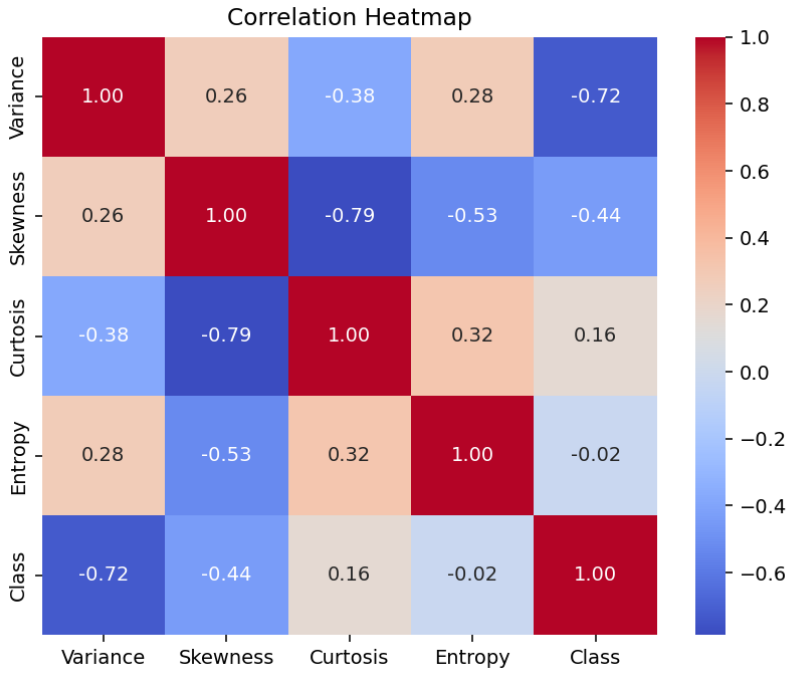

Figure 2: Correlation Heatmap of Features

Interpretation: Skewness and Curtosis show a fairly strong negative correlation with each other, while Variance and Curtosis are also negatively correlated. Entropy does not correlate strongly with any of the other three features, which suggests it carries somewhat independent information.

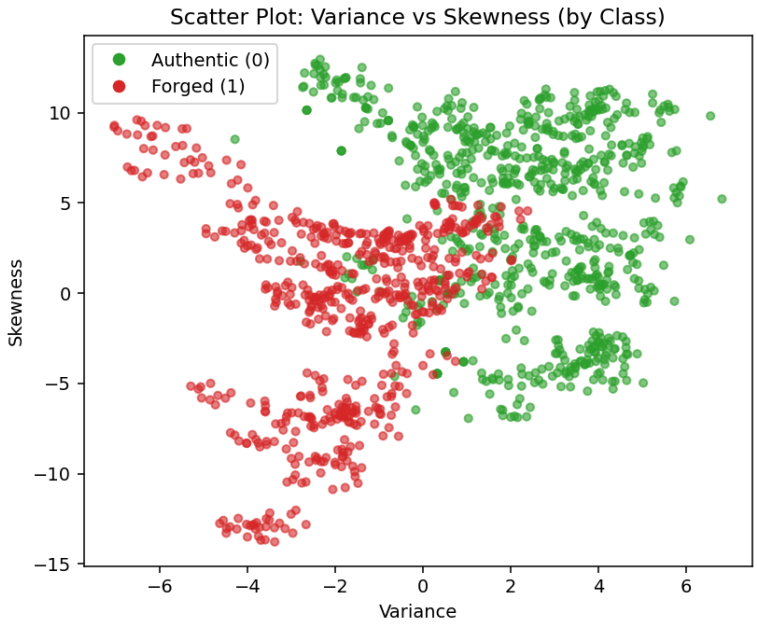

Figure 3: Scatter Plot of Variance vs Skewness, coloured by Class

Interpretation: The two classes (authentic in green, forged in red) form fairly distinguishable clusters in this 2D projection, with authentic notes generally having higher Variance and Skewness values. This roughly linear separation is a good sign that a perceptron should be able to do a decent job on this dataset.

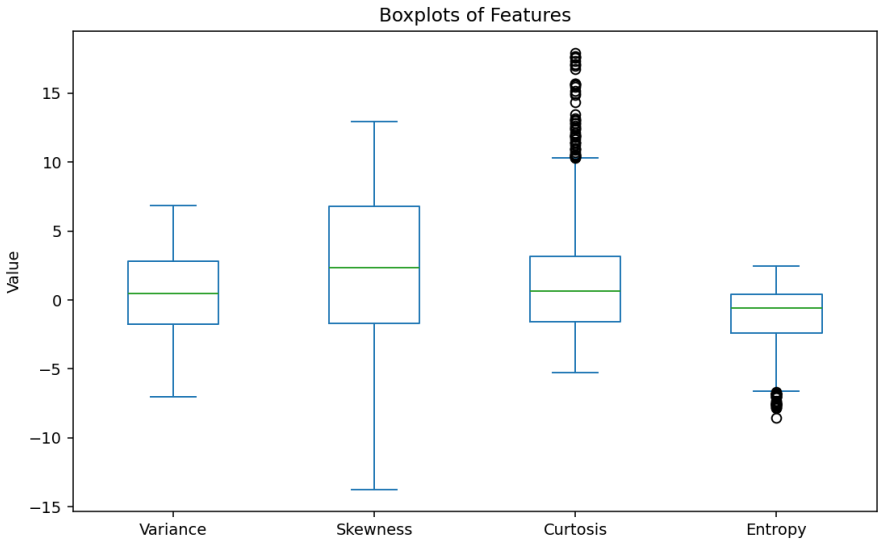

Figure 4: Boxplots of the Four Features

Interpretation: Skewness and Curtosis show a good number of outlier points beyond the whiskers, while Variance and Entropy are comparatively more compact. This matches what we already saw in the histograms.

5.3 Task 3: Data Preprocessing

All four numerical features were standardized using `StandardScaler` so that each feature has zero mean and unit variance (this matters a lot for perceptron convergence since the raw features here are on very different scales). The data was then split into 80% training and 20% testing sets.

```
x = df.iloc[:, :-1]
y = df.iloc[:, -1]

X = x.values
Y = y.values

from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
X = sc.fit_transform(X)

from sklearn.model_selection import train_test_split
X_train, X_test, Y_train, Y_test = train_test_split(
    X, Y, test_size=0.2, random_state=0)
```

5.4 Task 4: Perceptron Implementation

The perceptron was implemented from scratch as a Python class with weight initialization, bias initialization, the step activation function, forward propagation and the perceptron learning rule (weight/bias update).

```
class Perceptron:

    def __init__(self, weights, bias):
        self.weight = weights
        self.bias = bias
```

```
    def weightedsum(self, weights, input, bias):
        weightedsum = 0
        for i in range(len(weights)):
            weightedsum = weightedsum + (weights[i] * input[i])
        weightedsum = weightedsum + bias
        return weightedsum

    def activation(self, weightedsum):
        if weightedsum > 0:
            return 1
        else:
            return 0

    def predict(self, inputs):
        weightedsum = self.weightedsum(self.weight, inputs, self.bias)
        return self.activation(weightedsum)

    def backwardpass(self, input, target, learningrate):
        predicted = self.predict(input)
        error = target - predicted
        for i in range(len(self.weight)):
            self.weight[i] = self.weight[i] + learningrate * error *
                input[i]
        self.bias = self.bias + learningrate * error
```

5.5 Task 5: Model Training

The perceptron was trained for 20 epochs with a learning rate of 0.1, starting from zero weights and zero bias. After every epoch, the number of misclassified samples, the updated weights and the updated bias were recorded.

```
learning_rate = 0.1
epochs = 20
p = Perceptron([0, 0, 0, 0], 0)

error_history = []
weight_history = []
bias_history = []

for epoch in range(epochs):
    errors = 0
    for i in range(len(X_train)):
        prediction = p.predict(X_train[i])
        if prediction != Y_train[i]:
            errors += 1
        p.backwardpass(X_train[i], Y_train[i], learning_rate)

    error_history.append(errors)
    weight_history.append(p.weight.copy())
    bias_history.append(p.bias)

    print("Epoch", epoch + 1)
    print("Errors:", errors)
    print("Weights:", p.weight)
    print("Bias:", p.bias)
```

Table 4 shows the epoch-wise learning behaviour of the perceptron (errors reduced from 52 in the first epoch down to 15 by the 20th epoch).

Table 4: Epoch-wise Learning (selected epochs)

Epoch	Errors	Weight 1	Weight 2	Bias
1	52	-0.5372	-0.8095	-0.2000
2	26	-0.7590	-0.9752	-0.2000
5	23	-0.9806	-1.2693	-0.3000
10	16	-1.2140	-1.5917	-0.5000
15	19	-1.3188	-1.7007	-0.4000
17	17	-1.3116	-1.7443	-0.5000
20	15	-1.3636	-1.7644	-0.6000

5.6 Task 6: Model Evaluation

The trained perceptron was evaluated on the held-out test set using accuracy, precision, recall, F1-score and the confusion matrix.

```
from sklearn.metrics import (accuracy_score, confusion_matrix,
                             precision_score, recall_score, f1_score)

print("Accuracy:", accuracy_score(Y_test, predictions))
print("Confusion Matrix:")
print(confusion_matrix(Y_test, predictions))
print("Precision:", precision_score(Y_test, predictions))
print("Recall:", recall_score(Y_test, predictions))
print("F1 Score:", f1_score(Y_test, predictions))
```

Table 5: Performance of the Scratch-Built Perceptron

Metric	Value
Accuracy	0.9818
Precision	0.9593
Recall	1.0000
F1-score	0.9793

The confusion matrix obtained was $\begin{bmatrix} 152 & 5 \\ 0 & 118 \end{bmatrix}$, meaning all forged notes in the test set were correctly identified (no false negatives), while 5 authentic notes were wrongly flagged as forged.

5.7 Task 7: Effect of Different Learning Rates

The training was repeated with learning rates 0.01, 0.1 and 0.5 to study how the choice of η affects convergence.

```
learning_rates = [0.01, 0.1, 0.5]

for lr in learning_rates:
    p = Perceptron([0, 0, 0, 0], 0)
    error_history = []
    for epoch in range(epochs):
        errors = 0
```

```
        for i in range(len(X_train)):
            prediction = p.predict(X_train[i])
            if prediction != Y_train[i]:
                errors += 1
            p.backwardpass(X_train[i], Y_train[i], lr)
        error_history.append(errors)
    plt.plot(range(1, epochs + 1), error_history, label="LR=" + str(lr))

plt.title("Learning Rate Comparison")
plt.xlabel("Epoch")
plt.ylabel("Errors")
plt.legend()
plt.show()
```

All three learning rates converged to the same final test accuracy of 0.9818, though the epoch-wise error trajectory differed slightly across them (see Figure 9).

5.8 Task 8 (Optional): Comparison with Scikit-learn's Perceptron

Scikit-learn's built-in `Perceptron` class was trained with the same hyperparameters (20 iterations, $\eta = 0.1$) for a direct comparison.

```
from sklearn.linear_model import Perceptron

model = Perceptron(max_iter=20, eta0=0.1, random_state=42, tol=None)
model.fit(X_train, Y_train)
prediction = model.predict(X_test)

print("Accuracy:", accuracy_score(Y_test, prediction))
```

Table 6: Scratch Implementation vs Scikit-learn Perceptron

Metric	**Scratch Perceptron**	**Sklearn Perceptron**
Accuracy	0.9818	0.9891
Precision	0.9593	0.9832
Recall	1.0000	0.9915
F1-score	0.9793	0.9873

The scikit-learn implementation performed marginally better, most likely because of small differences in how it handles convergence internally, but the scratch perceptron gets extremely close, which is a good sanity check that the implementation is correct.

6 Mandatory Plots

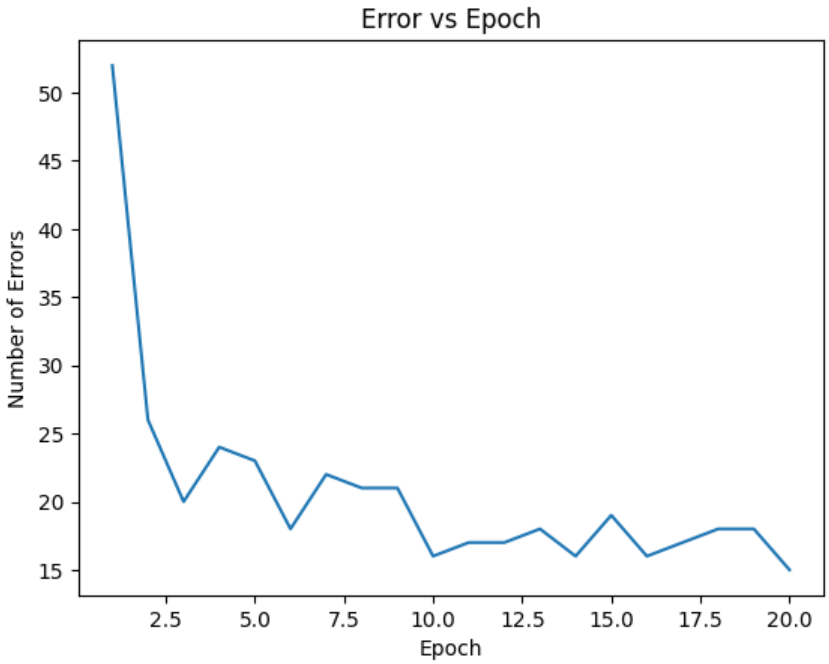

Figure 5: Training Error vs Epoch

Interpretation: The number of misclassified samples drops sharply within the first few epochs and then gradually flattens out, which shows the perceptron is converging towards a stable decision boundary rather than oscillating indefinitely.

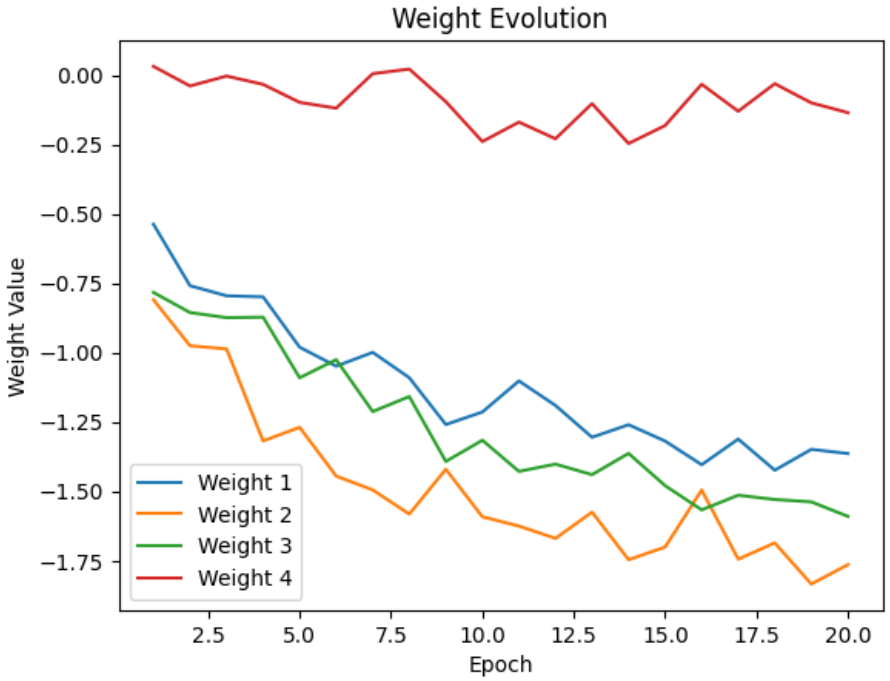

Figure 6: Weight Evolution over Epochs

Interpretation: All four weights move quickly away from zero in the initial epochs and

then settle into a fairly narrow band, indicating that the model has essentially learned the relative importance of each feature by the later epochs.

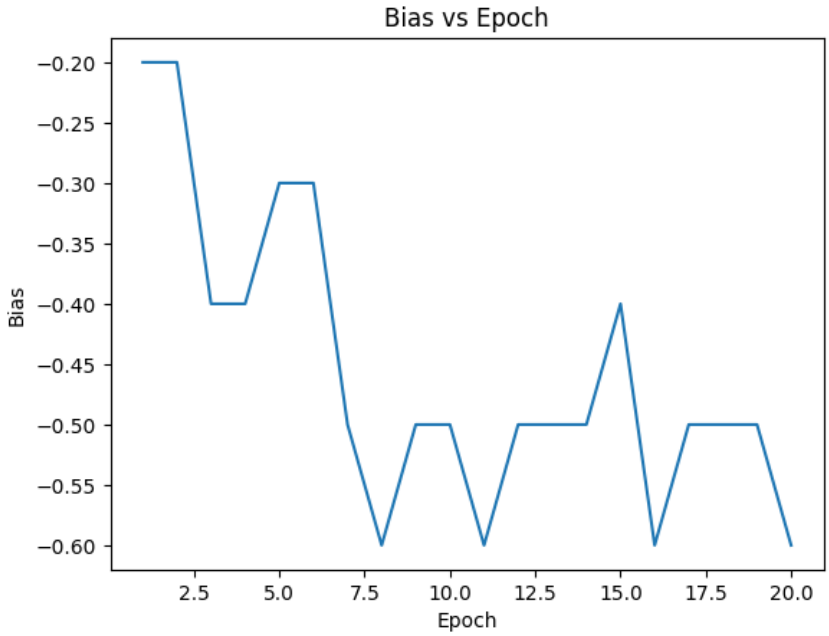

Figure 7: Bias Evolution over Epochs

Interpretation: The bias moves in discrete steps (since updates only happen on misclassification) and settles around -0.6, effectively shifting the decision boundary to better separate the two classes.

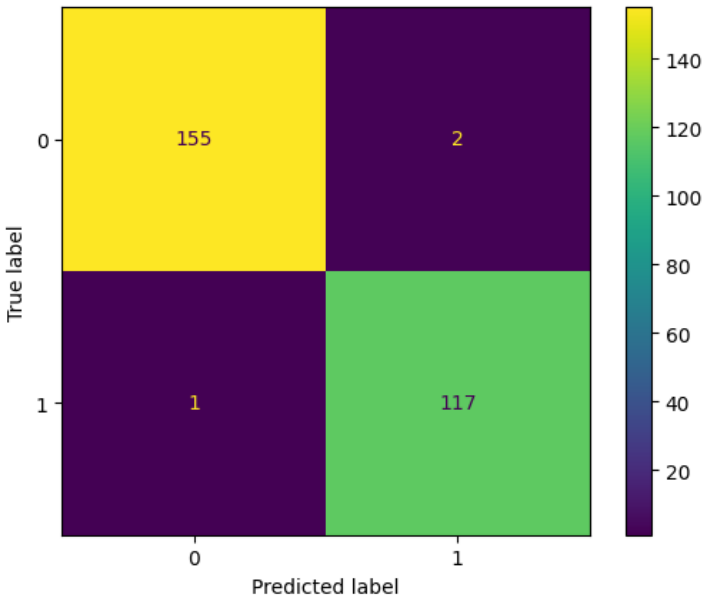

Figure 8: Confusion Matrix (Scikit-learn Perceptron)

Interpretation: The confusion matrix shows only 3 misclassifications out of 275 test samples, confirming the high accuracy reported earlier.

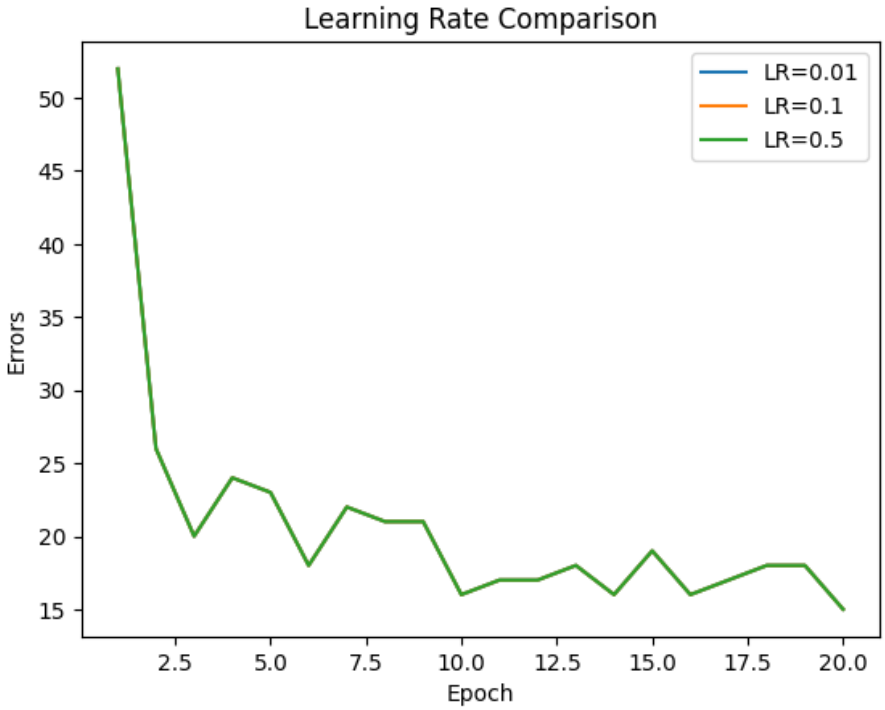

Figure 9: Learning Rate Comparison

Interpretation: A higher learning rate causes larger, noisier swings in the error count from epoch to epoch, whereas the smaller learning rate converges more smoothly, even though all three eventually reach a similar final accuracy.

7 Additional Tasks

7.1 Step vs Sigmoid Activation

The Step function only ever gives a hard 0 or 1 output and has zero gradient everywhere except at $z = 0$, where it is undefined. This makes it impossible to use with gradient-based optimization (backpropagation), since there is no useful gradient signal to propagate backward through the network. The Sigmoid function, on the other hand, is smooth and differentiable everywhere, produces a continuous output between 0 and 1, and gives a well-defined gradient that can be used to update weights via gradient descent. This is exactly why modern deep networks use differentiable activations like Sigmoid, Tanh or ReLU instead of the Step function.

7.2 Comparison with Scikit-learn's Perceptron

Already discussed in Section 6.8 above – the scratch implementation and the scikit-learn implementation produce very similar results, with sklearn's version edging out slightly higher accuracy and precision.

7.3 Learning Rate Experiments

Already discussed in Section 6.7 – learning rates of 0.01, 0.1 and 0.5 were all tried and all converged to the same final test accuracy, though the smaller learning rate produced a smoother error curve.

7.4 Why a Single Layer Perceptron Cannot Solve XOR

A single layer perceptron can only draw a single straight line (or hyperplane in higher dimensions) to separate the two classes. The XOR function, however, is not linearly separable – there is no single straight line that can separate the $(0,0) \to 0$, $(1,1) \to 0$ points from the $(0,1) \to 1$, $(1,0) \to 1$ points, since these two groups are diagonally opposite to each other. To solve XOR, at least one hidden layer is required, which is exactly the motivation behind multilayer perceptrons.

7.5 Effect of Feature Normalization on Convergence

Before normalization, the four features here are on very different scales (Curtosis alone ranges up to about 18, while Entropy stays under 3), which would cause the gradient updates to be dominated by whichever feature happens to have the largest raw magnitude. After applying `StandardScaler`, all features are brought to a comparable scale (zero mean, unit variance), which lets the perceptron treat all four features fairly during training and converge in far fewer epochs than it would on the raw, unscaled data.

8 Discussion

Overall, the scratch-built perceptron performed extremely well on this dataset, reaching close to 98% accuracy on unseen test data. This is largely because the Banknote Authentication dataset is (as seen in the scatter plot) reasonably close to being linearly separable in feature space, which is exactly the kind of problem the classical perceptron is designed for. The comparison with scikit-learn's Perceptron class confirmed that the from-scratch implementation of the weighted sum, step activation and the weight/bias update rule was correct. The learning-rate study showed that convergence speed and stability are sensitive to η, even though the final accuracy in this case did not change much across the tested values.

9 Conclusion

This experiment gave a solid, hands-on understanding of how a single artificial neuron learns from data using the perceptron learning rule. Implementing the perceptron from scratch made the weighted sum, step activation and weight/bias update equations much more concrete than just reading them off a page. The evaluation metrics and the near-identical performance compared to scikit-learn's implementation both confirm that the perceptron was implemented and trained correctly. At the same time, the XOR discussion is a good reminder of exactly where the single layer perceptron's capability ends, which naturally motivates the multilayer perceptron that will be covered in the next experiment.

10 References

[1] F. Rosenblatt, "The Perceptron," *Psychological Review*, 1958.

[2] I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.

[3] C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.

[4] S. Haykin, *Neural Networks and Learning Machines*, Pearson, 2009.

[5] UCI Machine Learning Repository – Banknote Authentication Dataset.

[6] Scikit-learn Documentation: Perceptron.